



Lecture 21: Reusable Markup with Snippets

INTERVIEW QUESTION

[> CORE](#)

How do you write a reusable chunk of markup you can use in several places?

The same three lines of markup appear in five spots in your component. Copy-paste feels wrong. Can you name a block of markup and reuse it?

The Five-Copy Disaster

You are building a product page. The same `<figure>` block, an image with a caption and a price badge, has to appear in five spots: the hero gallery, the "Customers also bought" strip, the search preview, the "Recently viewed" shelf, and a share dialog. You copy the five lines into all five places and ship. Three weeks later the designer asks for a loading spinner on every image. You edit the copies, but one was hiding inside an `{#if}` branch you forgot existed, so the "Recently viewed" shelf still renders the old markup. There is no error and no red squiggle. The page just looks broken in one corner, and the only way to catch it is for a human to notice.

That is the failure mode of duplicated markup: the copies drift apart silently, and the cost grows with every copy. The fix is not more discipline. The fix is making sure there is only *one* copy that has a name.

Naming the Block

Notice what the five copies actually are. They are not five features. They are one block of HTML stamped into the page five times, each time with different data. So the job is this: take a block of markup, give it a name, and call that name wherever the block should appear, passing in the data for that spot.

This is exactly what a function does for JavaScript: name a block of statements once, call it by name with arguments. Svelte gives you the same move for markup. That named, callable block of template is called a **snippet**. You define it with `{#snippet ...}` and stamp it into the page with `{@render ...}`.

KEY TAKEAWAY

A snippet is to markup what a function is to logic: name it once, call it anywhere.

Just as a function lets you write a block of JavaScript once and invoke it with different arguments, a snippet lets you write a block of HTML once and render it with different data. The markup lives in exactly one place, so a change to the snippet updates every spot that renders it, with zero copies to hunt down.

```
1  {#snippet figure()}
```

1. **NAME** *the block of markup once*

```
2    <figure>
```

```
3      
```

```
4      <figcaption>Flagship smartphone</figcaption>
```

```
5    </figure>
```

```
6  {/snippet}
```

```
7
```

```
8  {@render figure()}
```

2. **CALL** *it by name, as many times as you need*

```
9  {@render figure()}
```

Passing Parameters

A snippet that renders one fixed image is barely better than a copy. Its real power appears when you give it **parameters**, named inputs exactly like function arguments, so each call site passes its own data.

● ● ● ProductPage.svelte

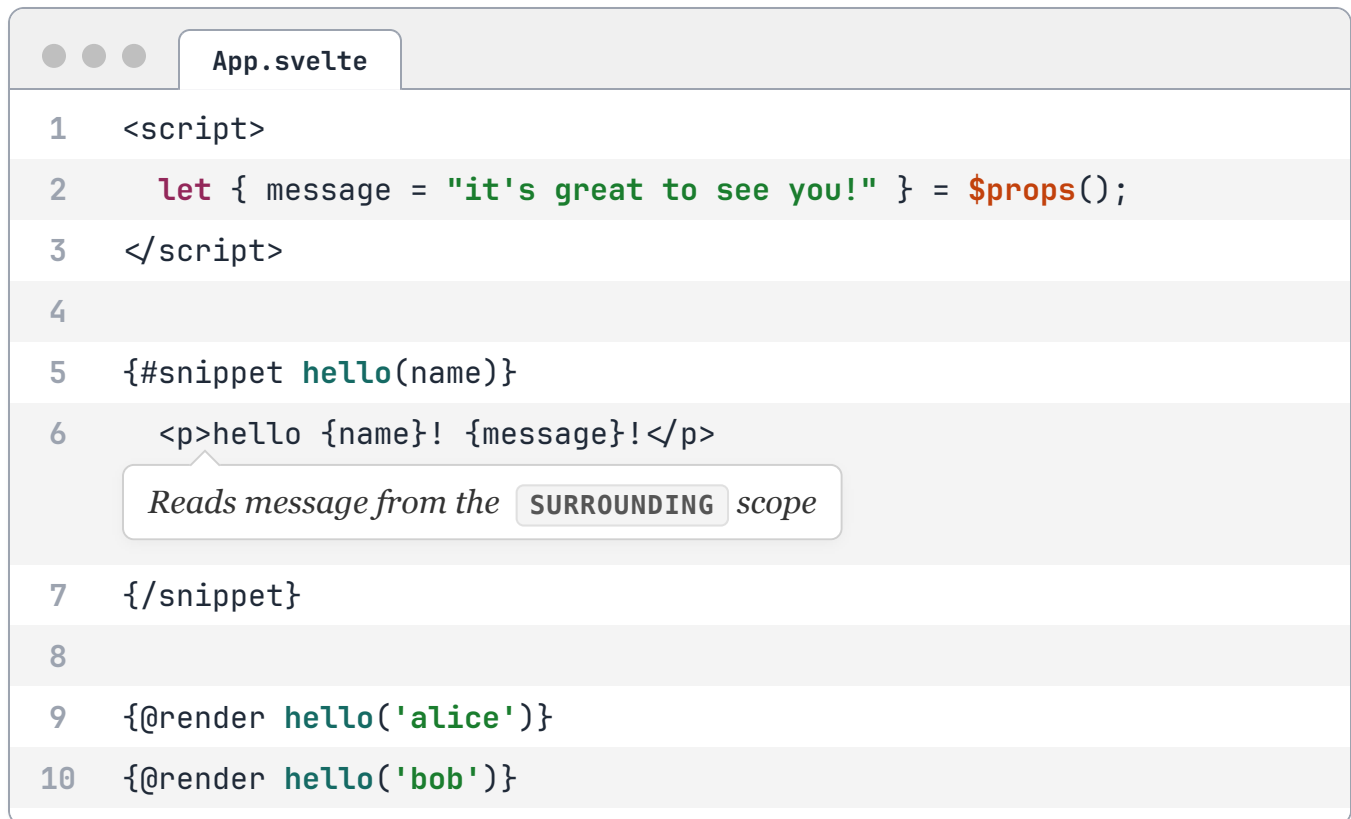
```
1  {#snippet figure(image)}  
    The PARAMETER lets each call pass its own data  
2      <figure>  
3          <img src={image.src} alt={image.caption} />  
4          <figcaption>{image.caption}</figcaption>  
5      </figure>  
6  {/snippet}  
7  
8  {#each products as product}  
9      {#if product.href}  
10         <a href={product.href}>{@render figure(product)}</a>  
    Linked products wrap the figure  
11         { :else }  
12         {@render figure(product)}  
    Without a snippet this <figure> would be DUPLICATED here  
13         { /if }  
14     { /each }
```

Every product renders the same `<figure>`, but linked products wrap it in an `<a>` while unlinked ones render it bare. Without a snippet you would duplicate the entire `<figure>` in both branches, giving you two copies guaranteed to drift. Like a

function, a snippet can have any number of parameters with default values and destructuring; the one thing you cannot use is a rest parameter.

Snippet Scope

Snippets obey ordinary **lexical scope**: they read any variable visible where they are declared, whether from the `<script>` tag or a surrounding `{#each}` block, with no passing required. They are visible to siblings and their children, but not outside the block they were declared in, and they can even call themselves, so recursive markup is possible.

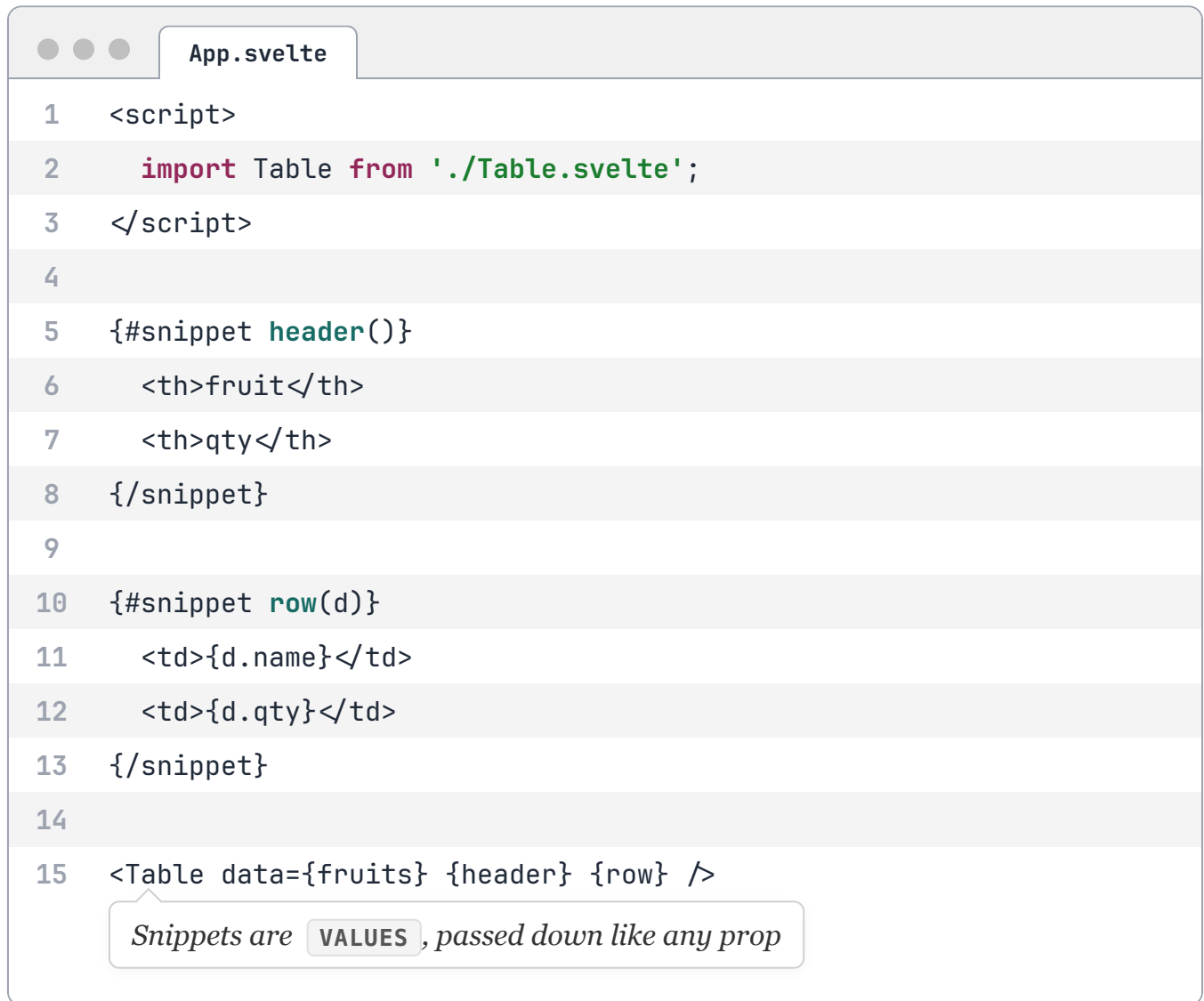


```
1  <script>
2    let { message = "it's great to see you!" } = $props();
3  </script>
4
5  {#snippet hello(name)}
6    <p>hello {name}! {message}!</p>
7  {/snippet}
8
9  {@render hello('alice')}
10 {@render hello('bob')}
```

*Reads message from the **SURROUNDING** scope*

Snippets as Component Props

So far the snippet has stayed inside the component that declared it. The bigger unlock is that a snippet is a *value*, just like a number or a string, so you can hand it to a child as a prop and let the child decide when and where to render it. You are passing *content* down, not *data*.



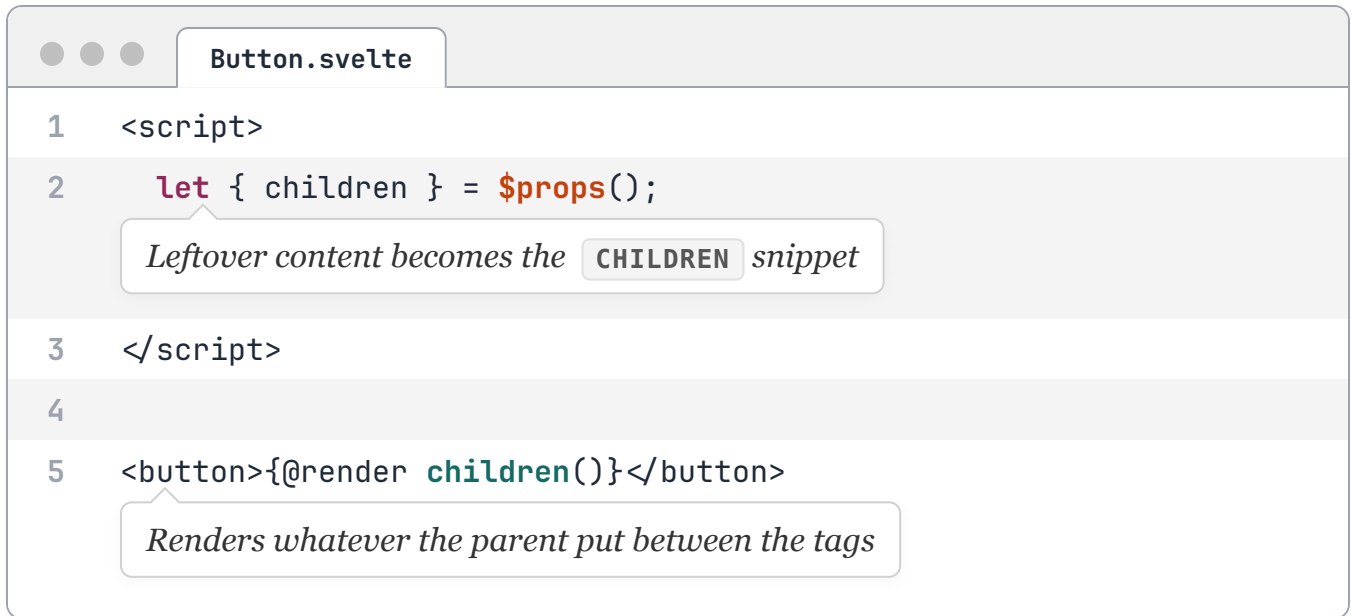
```
1  <script>
2    import Table from './Table.svelte';
3  </script>
4
5  {#snippet header()}
6    <th>fruit</th>
7    <th>qty</th>
8  {/snippet}
9
10 {#snippet row(d)}
11   <td>{d.name}</td>
12   <td>{d.qty}</td>
13 {/snippet}
14
15 <Table data={fruits} {header} {row} />
```

Snippets are **VALUES**, passed down like any prop

The child receives the snippets through `$props()` and renders them, feeding each row of data into the `row` snippet as it loops (`<tr>{@render row(d)}</tr>`). As a convenience, a snippet declared directly *between a component's tags* becomes a prop on that component automatically, so the attribute syntax is optional.

The `children` Snippet and the End of Slots

Any content between a component's tags that is *not* a named snippet automatically becomes a special snippet called `children`, which is what powers the most common pattern in Svelte: a wrapper that renders whatever you put inside it.



```
1 <script>
2   let { children } = $props();
3 </script>
4
5 <button>{@render children()}</button>
```

*Leftover content becomes the **CHILDREN** snippet*

Renders whatever the parent put between the tags

Because a snippet prop can be missing, render optional ones defensively with `{@render children?.() }`, or wrap them in an `{#if children}` block with fallback content in the `:else`.

KEY TAKEAWAY

Snippets are the replacement for slots: same job, but a snippet can take parameters and be passed around as a value.

In Svelte 4 you passed content into a component with **slots**, named placeholders a child filled with markup from its parent. Slots worked, but they could not take parameters and were not values you could store or pass. Snippets do everything slots did and more, so slots are deprecated in Svelte 5. One caution

from the slots era: because non-snippet content becomes the `children` snippet, a component cannot also declare a normal prop named `children` , so avoid that prop name.

INTERVIEW TIP

To impress the interviewer: Many candidates can recite that "slots are replaced by snippets." The answer that signals real depth is explaining *what snippets add*: because a snippet is a first-class value that takes parameters, one component can receive several differently-shaped blocks of markup (`header` , `row` , `footer`) and call each with its own data, something slots could never express. That is the mechanical reason the framework made the switch.

Summary

A **snippet** is a named, reusable block of markup inside a component, declared with `{#snippet name()}` and stamped into the page with `{@render name()}`. Snippets take parameters like functions, obey ordinary lexical scope, and are first-class values you can pass into child components as props, where leftover content becomes the special `children` snippet. By collapsing every duplicated copy of a block of markup into one named definition, snippets eliminate the silent drift that copy-paste causes and fully replace the deprecated slots mechanism from Svelte 4.

	COPY-PASTE / SLOTS (The Old Ways)	SNIPPETS (Svelte 5)
Single source	One copy per usage site, drifting silently	One named definition, rendered by name
Parameters	None (slots cannot take arguments)	Any number, with defaults and destructuring

	COPY-PASTE / SLOTS (The Old Ways)	SNIPPETS (Svelte 5)
Pass to child	Slots only, fixed by the child's template	As a prop value: <code>{header}</code> or between the tags
Reuse across files	Not possible	Export from <code><script module></code> (Svelte 5.5+)